

Deep Learning: Theory and Practice

Recurrent Neural Networks

28-03-2019

Introduction

- ❖ The standard DNN / CNN paradigms
 - ❖ $(\mathbf{x}, \mathbf{y})$ - ordered pair of data vectors / images ($\mathbf{x}$) and target ($\mathbf{y}$)
- ❖ Moving to sequence data
 - ❖ $(\mathbf{x}(t), \mathbf{y}(t))$ where this could be sequence to sequence mapping task.
 - ❖ $(\mathbf{x}(t), \mathbf{y})$ where this could be a sequence to vector mapping task.

Introduction

- ❖ Difference between CNNs / DNNs
 - ❖ $(x(t), y(t))$ where this could be sequence to sequence mapping task.
 - ❖ Input features / output targets are correlated in time.
 - ❖ Unlike standard models where each pair is independent.
 - ❖ Need to model dependencies in the sequence over time.

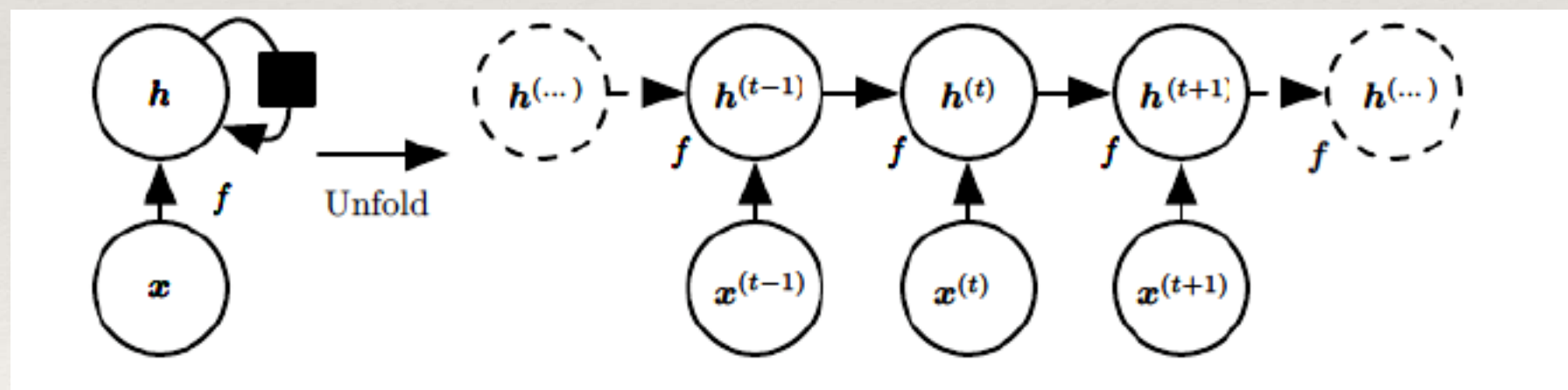
Introduction to Recurrent Networks

$$\mathbf{s}^{(t)} = f(\mathbf{s}^{(t-1)}; \boldsymbol{\theta}),$$

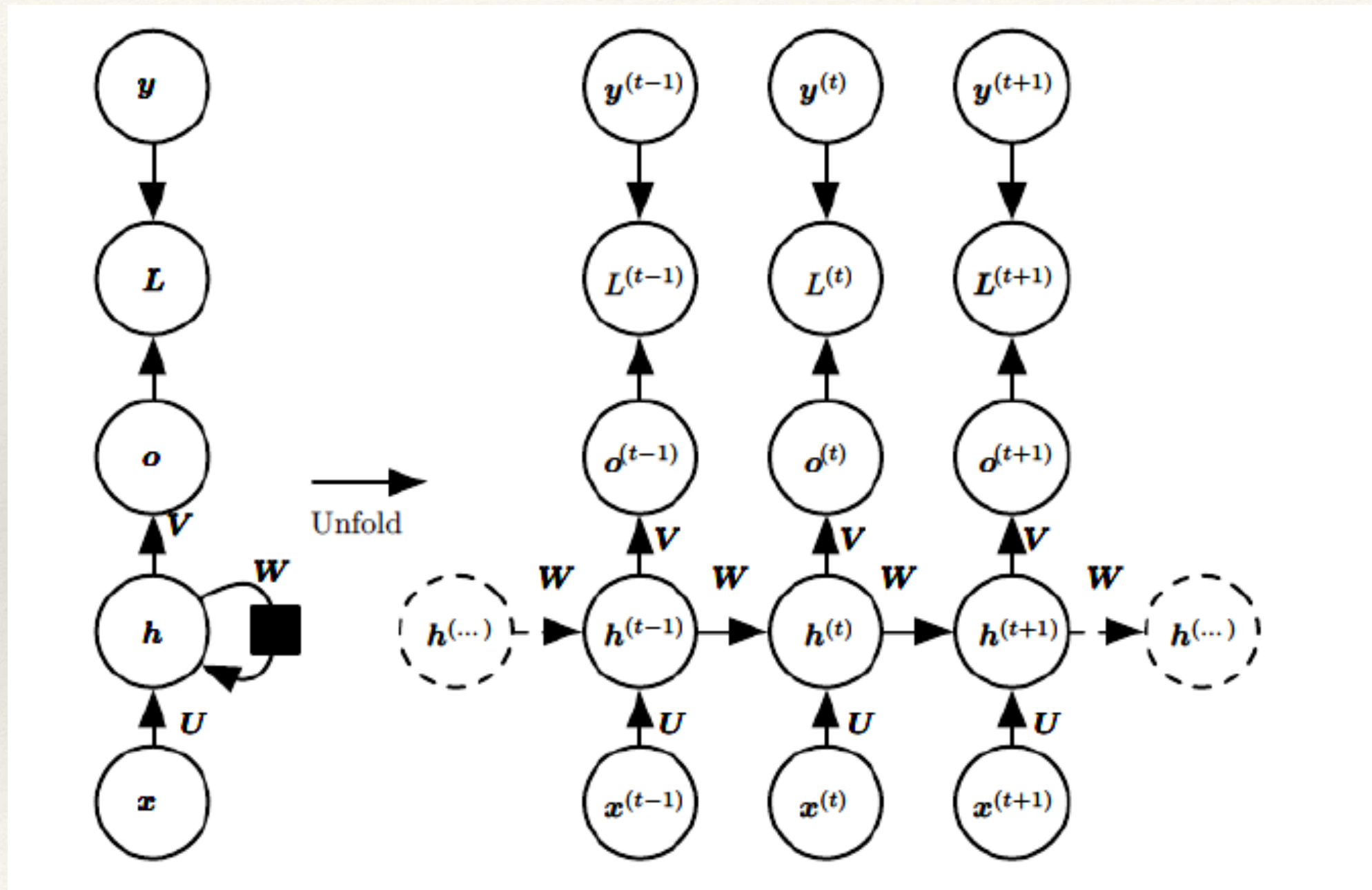
$$\begin{aligned}\mathbf{s}^{(3)} &= f(\mathbf{s}^{(2)}; \boldsymbol{\theta}) \\ &= f(f(\mathbf{s}^{(1)}; \boldsymbol{\theta}); \boldsymbol{\theta})\end{aligned}$$

$$\mathbf{s}^{(t)} = f(\mathbf{s}^{(t-1)}, \mathbf{x}^{(t)}; \boldsymbol{\theta}),$$

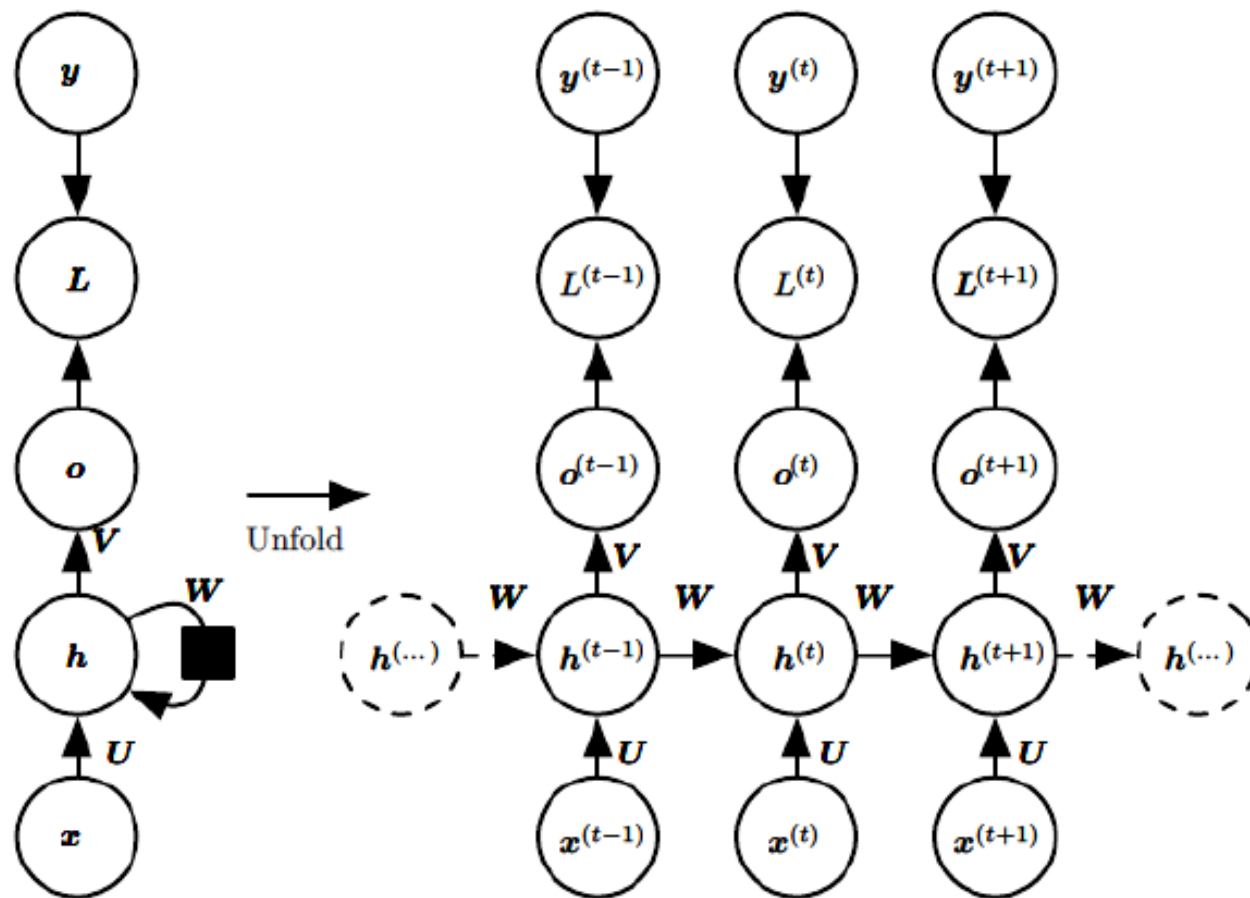
$$\mathbf{h}^{(t)} = f(\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}; \boldsymbol{\theta}),$$



Recurrent Networks



Recurrent Networks



$$\begin{aligned} \mathbf{a}^{(t)} &= \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)} \\ \mathbf{h}^{(t)} &= \tanh(\mathbf{a}^{(t)}) \\ \mathbf{o}^{(t)} &= \mathbf{c} + \mathbf{V}\mathbf{h}^{(t)} \\ \hat{\mathbf{y}}^{(t)} &= \text{softmax}(\mathbf{o}^{(t)}) \end{aligned}$$

$$\begin{aligned} &L\left(\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(\tau)}\}, \{\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(\tau)}\}\right) \\ &= \sum_t L^{(t)} \\ &= - \sum_t \log p_{\text{model}}\left(\mathbf{y}^{(t)} \mid \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(t)}\}\right) \end{aligned}$$

Back Propagation in RNNs

$$\begin{aligned} \mathbf{a}^{(t)} &= \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)} \\ \mathbf{h}^{(t)} &= \tanh(\mathbf{a}^{(t)}) \\ \mathbf{o}^{(t)} &= \mathbf{c} + \mathbf{V}\mathbf{h}^{(t)} \\ \hat{\mathbf{y}}^{(t)} &= \text{softmax}(\mathbf{o}^{(t)}) \end{aligned}$$

Model Parameters

$$\mathbf{U}, \mathbf{V}, \mathbf{W}, \mathbf{b} \text{ and } \mathbf{c}$$

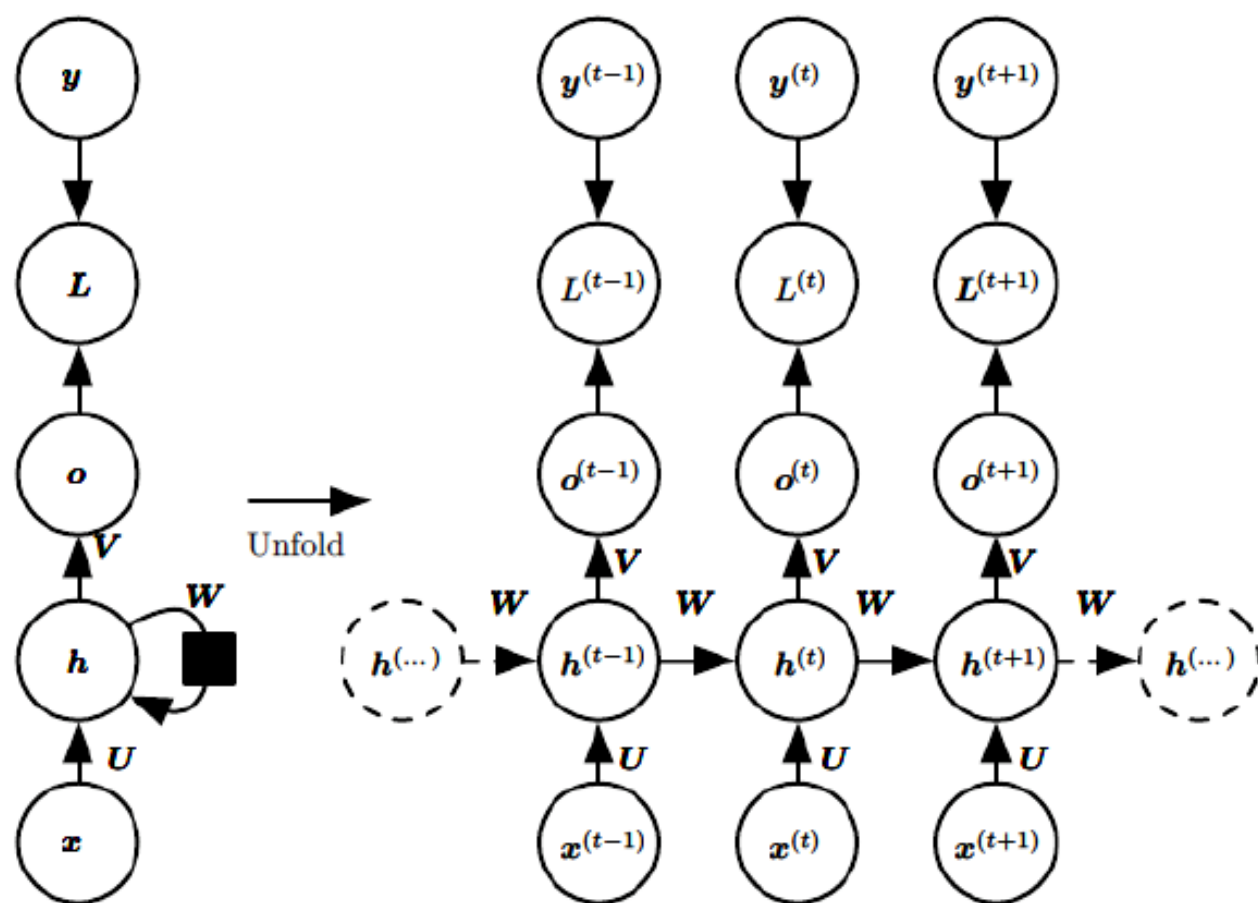
Gradient Descent

$$\begin{aligned} &L\left(\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(\tau)}\}, \{\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(\tau)}\}\right) \\ &= \sum_t L^{(t)} \\ &= - \sum_t \log p_{\text{model}}\left(y^{(t)} \mid \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(t)}\}\right) \end{aligned}$$

$$\frac{\partial L}{\partial L^{(t)}} = 1.$$

$$(\nabla_{\boldsymbol{\alpha}^{(t)}} L)_i = \frac{\partial L}{\partial o_i^{(t)}} = \frac{\partial L}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o_i^{(t)}} = \hat{y}_i^{(t)} - \mathbf{1}_{i, y^{(t)}}$$

Recurrent Networks

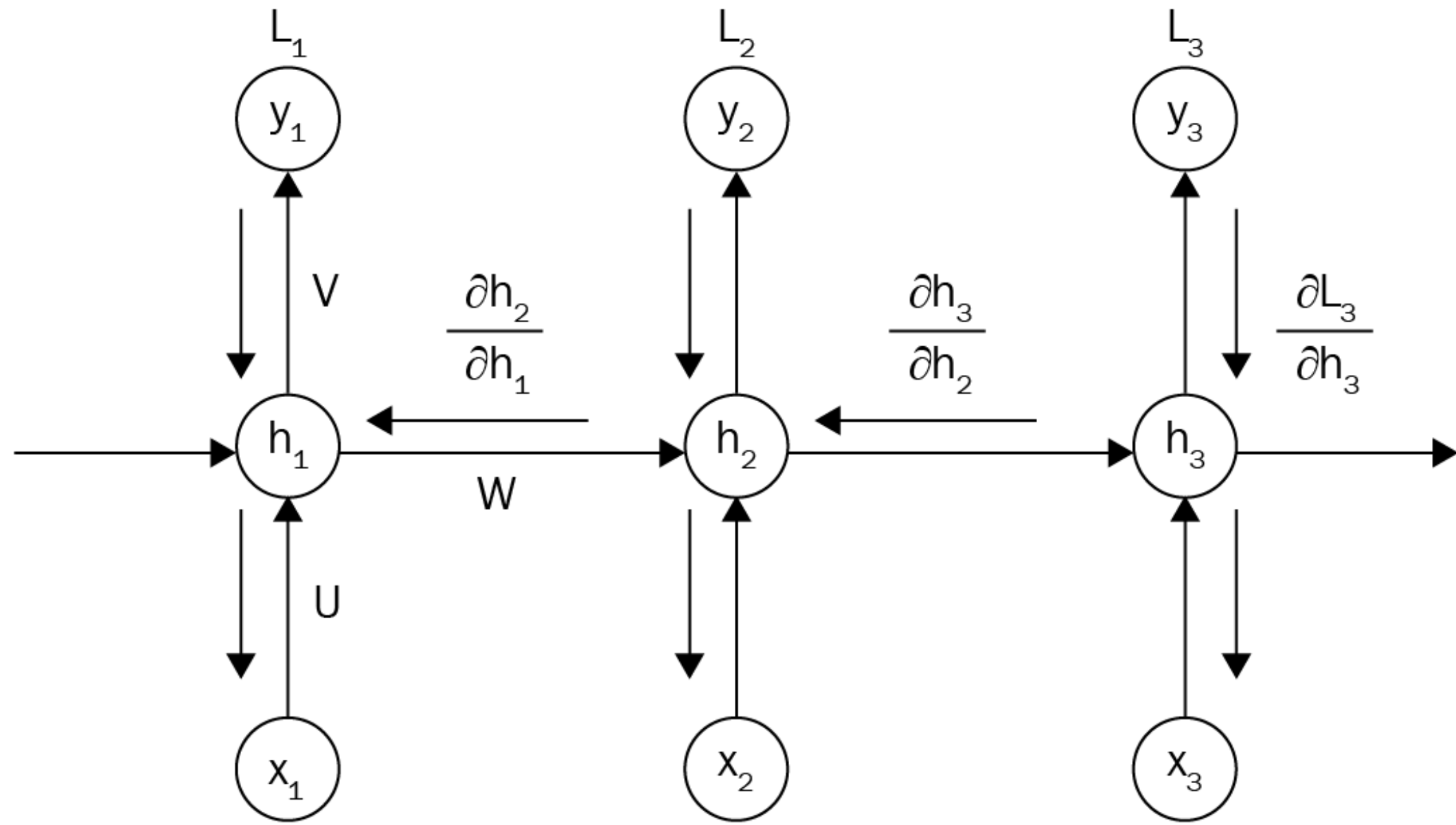


$$(\nabla_{\mathbf{o}^{(t)}} L)_i = \frac{\partial L}{\partial o_i^{(t)}} = \frac{\partial L}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o_i^{(t)}} = \hat{y}_i^{(t)} - \mathbf{1}_{i, y^{(t)}}$$

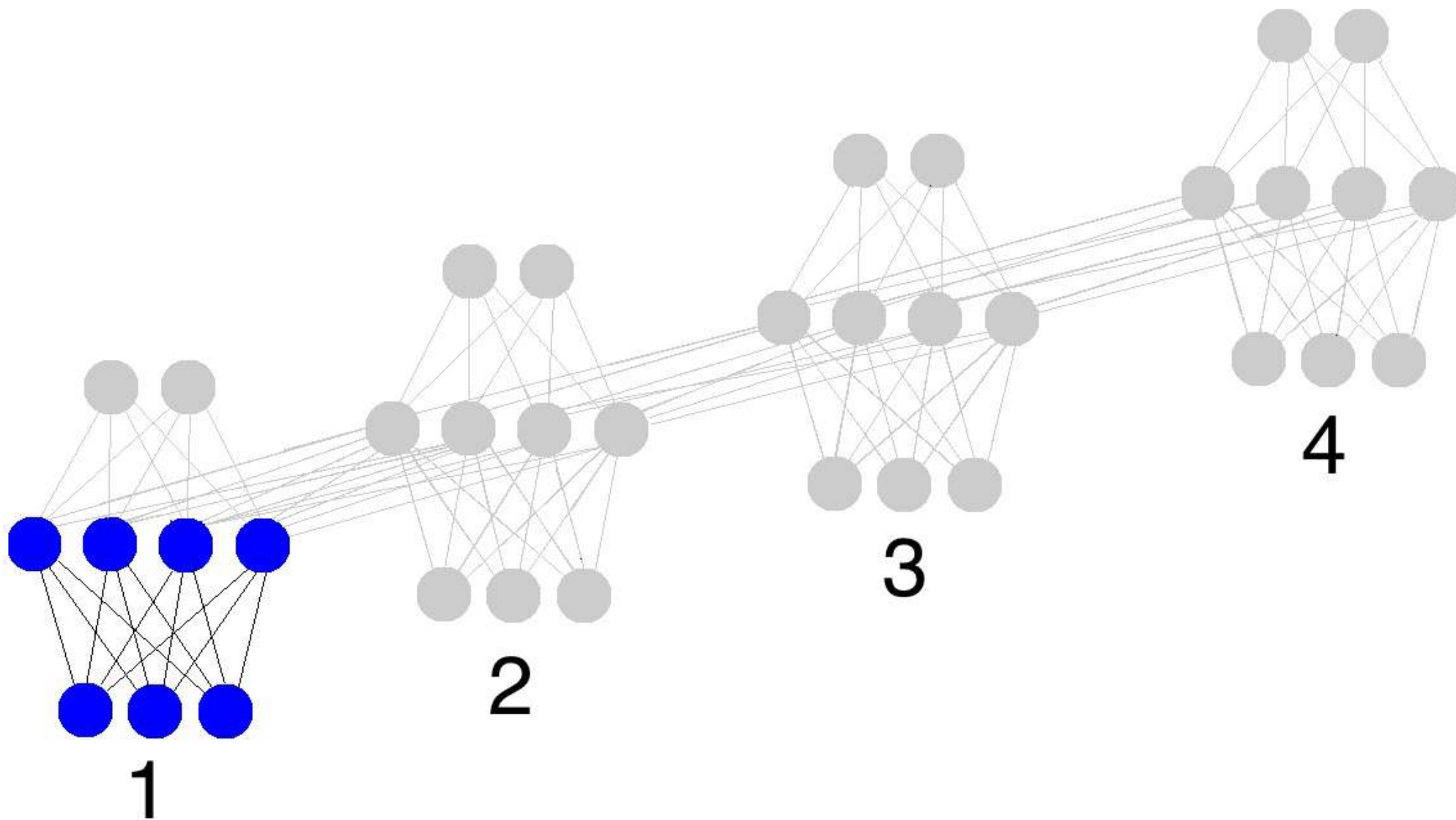
$$\nabla_{\mathbf{h}^{(\tau)}} L = \mathbf{V}^\top \nabla_{\mathbf{o}^{(\tau)}} L.$$

$$\begin{aligned} \nabla_{\mathbf{h}^{(t)}} L &= \left(\frac{\partial \mathbf{h}^{(t+1)}}{\partial \mathbf{h}^{(t)}} \right)^\top (\nabla_{\mathbf{h}^{(t+1)}} L) + \left(\frac{\partial \mathbf{o}^{(t)}}{\partial \mathbf{h}^{(t)}} \right)^\top (\nabla_{\mathbf{o}^{(t)}} L) \\ &= \mathbf{W}^\top (\nabla_{\mathbf{h}^{(t+1)}} L) \text{diag} \left(1 - \left(\mathbf{h}^{(t+1)} \right)^2 \right) + \mathbf{V}^\top (\nabla_{\mathbf{o}^{(t)}} L) \end{aligned}$$

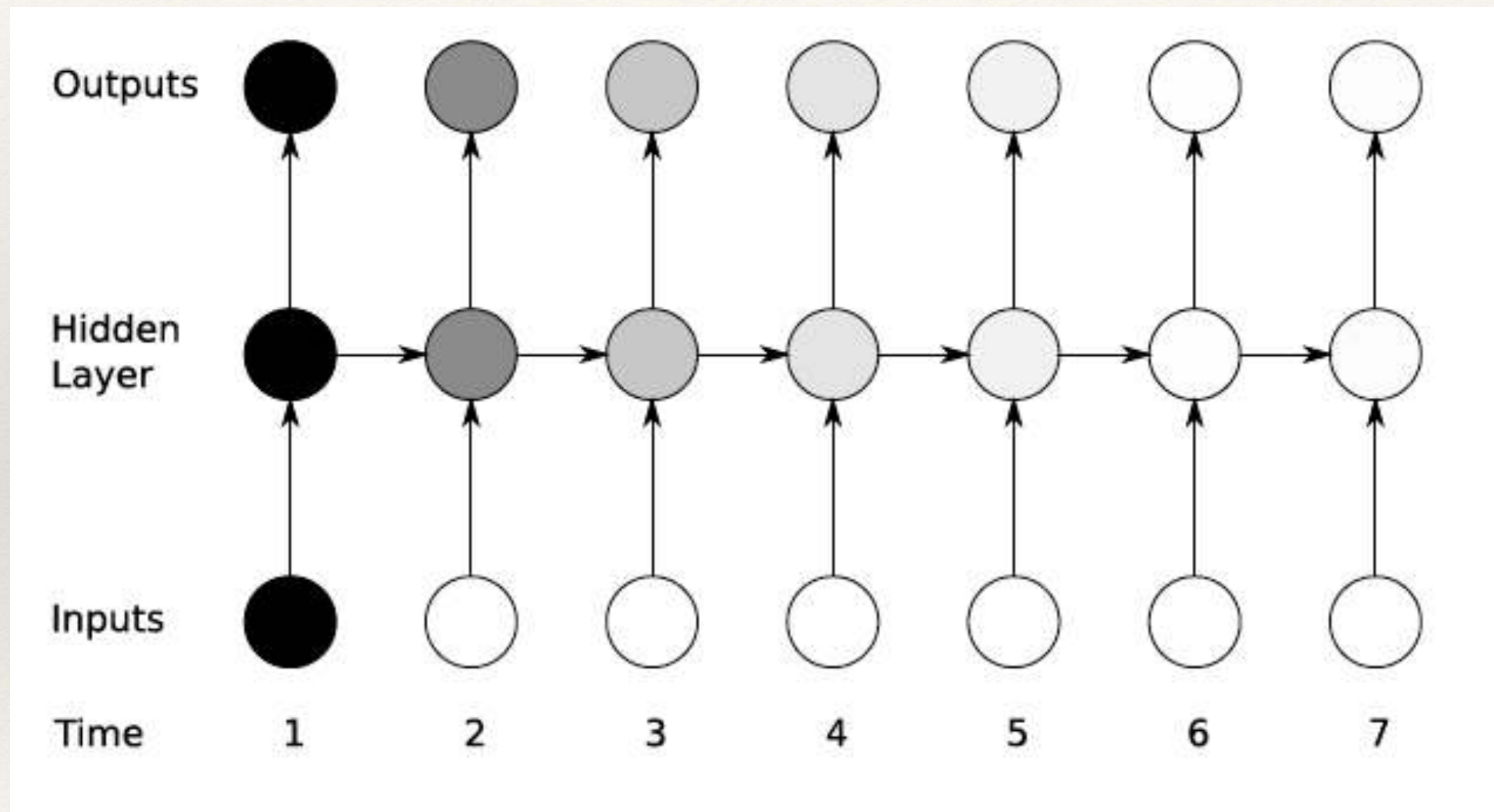
Back Propagation Through Time



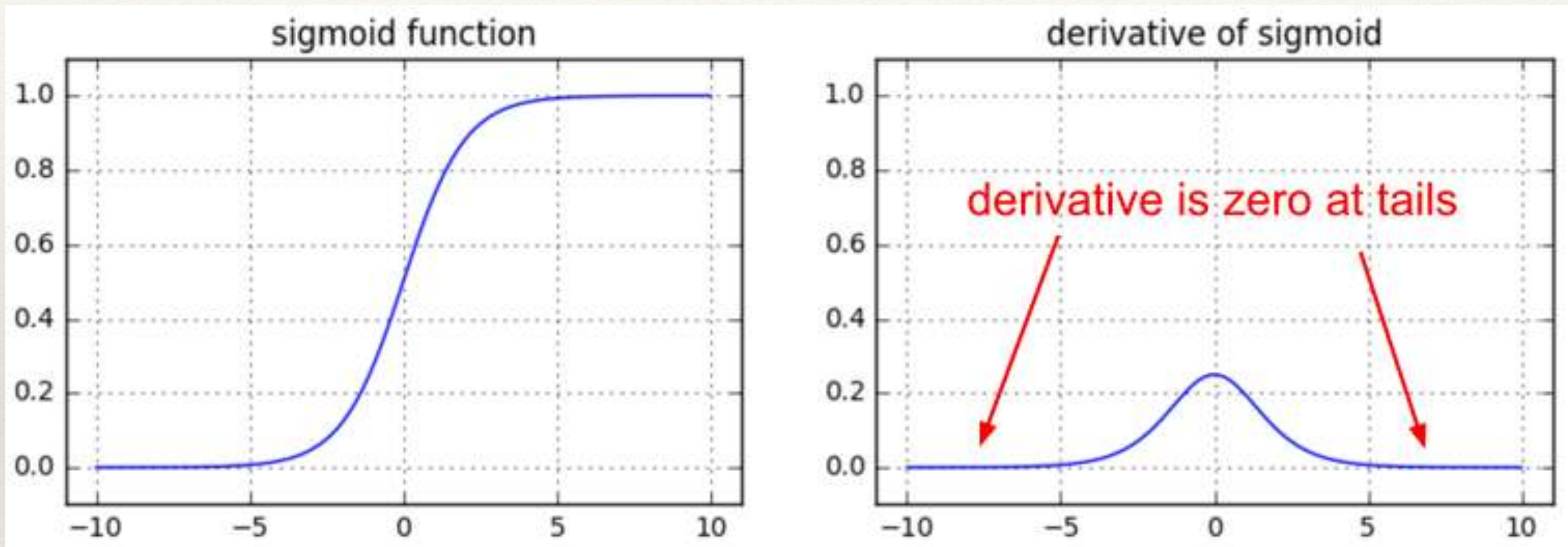
Back Propagation Through Time



Long-term Dependency Issues

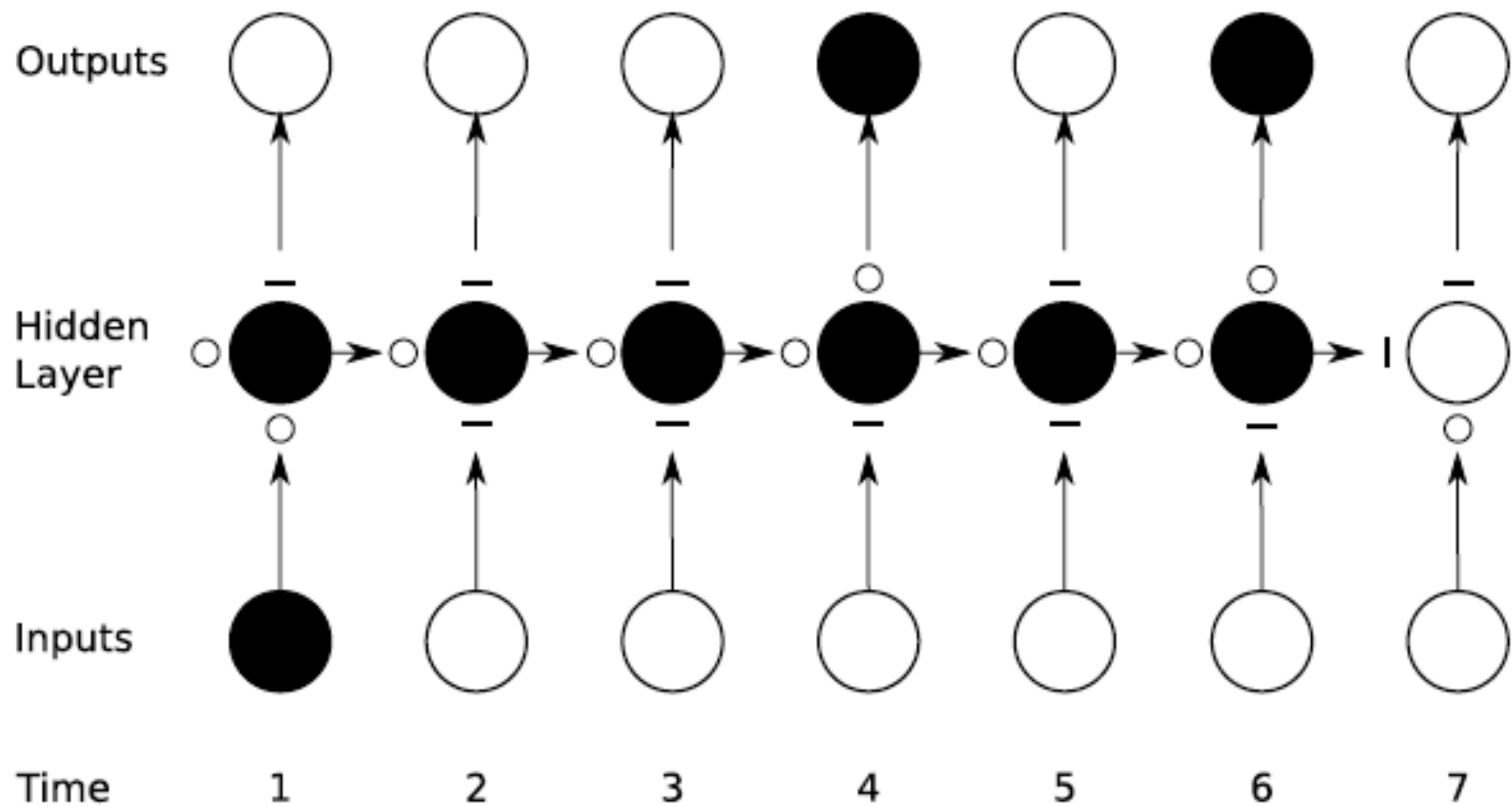


Vanishing/Exploding Gradients

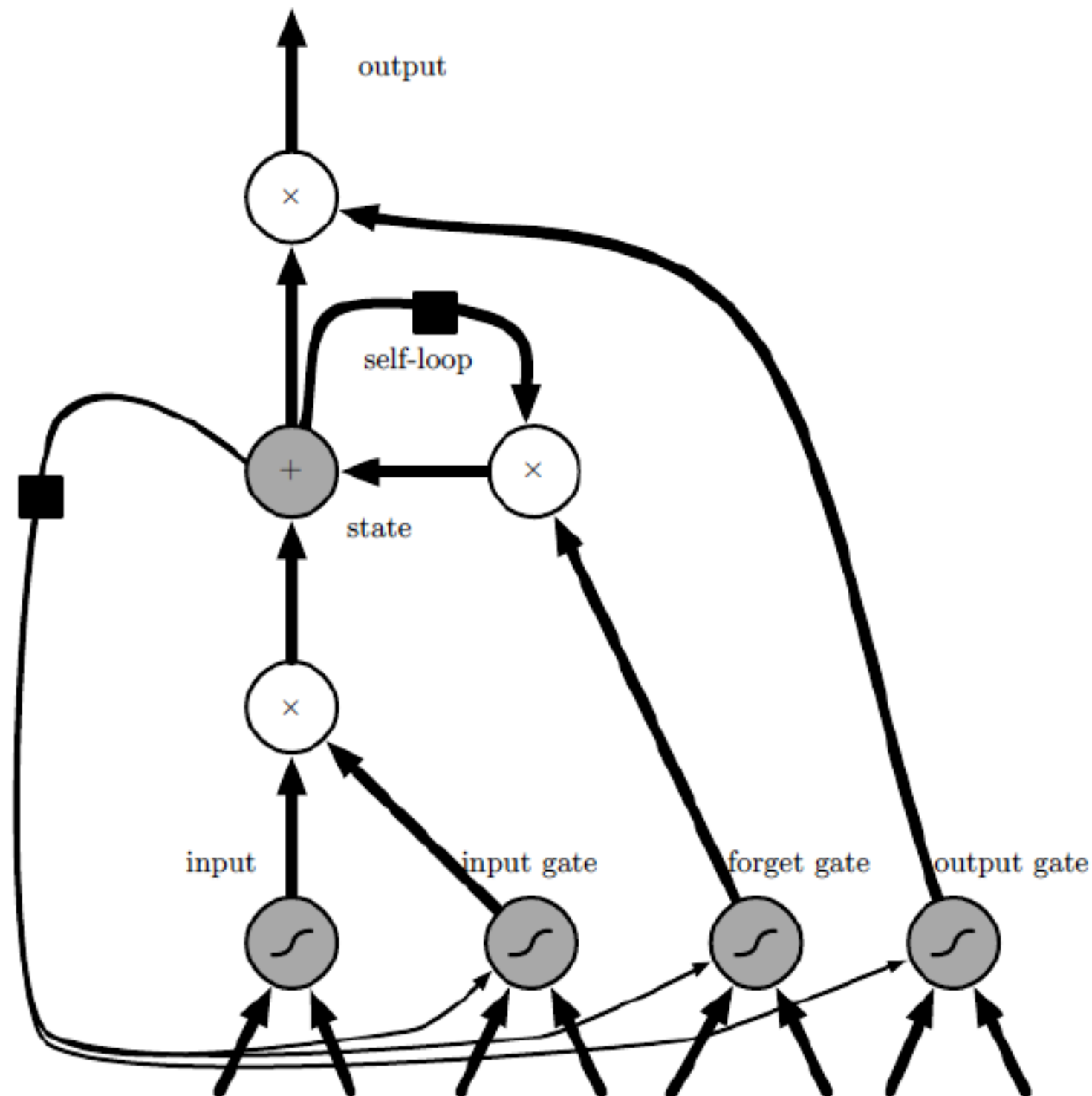


- ❖ Gradients either vanish or explode
 - ❖ Initial frames may not contribute to gradient computations or may contribute too much.

Long-Short Term Memory



Long Short Term Memory Networks



LSTM Cell

f - sigmoid function
g, h - tanh function

Input Gate

$$a_l^t = \sum_{i=1}^I w_{il} x_i^t + \sum_{h=1}^H w_{hl} b_h^{t-1} + \sum_{c=1}^C w_{cl} s_c^{t-1}$$
$$b_l^t = f(a_l^t)$$

Forget Gate

$$a_\phi^t = \sum_{i=1}^I w_{i\phi} x_i^t + \sum_{h=1}^H w_{h\phi} b_h^{t-1} + \sum_{c=1}^C w_{c\phi} s_c^{t-1}$$
$$b_\phi^t = f(a_\phi^t)$$

Cell

$$a_c^t = \sum_{i=1}^I w_{ic} x_i^t + \sum_{h=1}^H w_{hc} b_h^{t-1}$$
$$s_c^t = b_\phi^t s_c^{t-1} + b_l^t g(a_c^t)$$

Output Gate

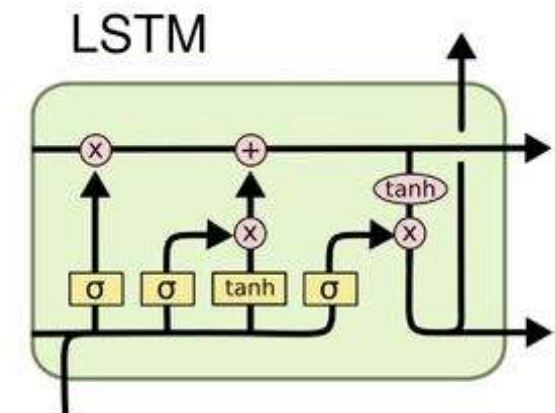
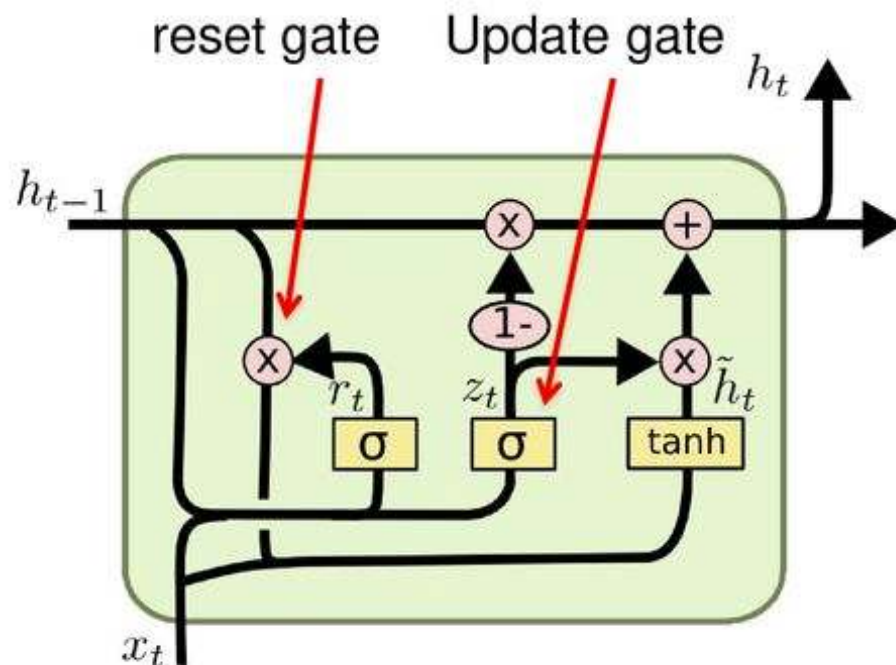
$$a_\omega^t = \sum_{i=1}^I w_{i\omega} x_i^t + \sum_{h=1}^H w_{h\omega} b_h^{t-1} + \sum_{c=1}^C w_{c\omega} s_c^t$$
$$b_\omega^t = f(a_\omega^t)$$

LSTM output

$$b_c^t = b_\omega^t h(s_c^t)$$

Gated Recurrent Units (GRU)

GRU – gated recurrent unit (more compression)



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

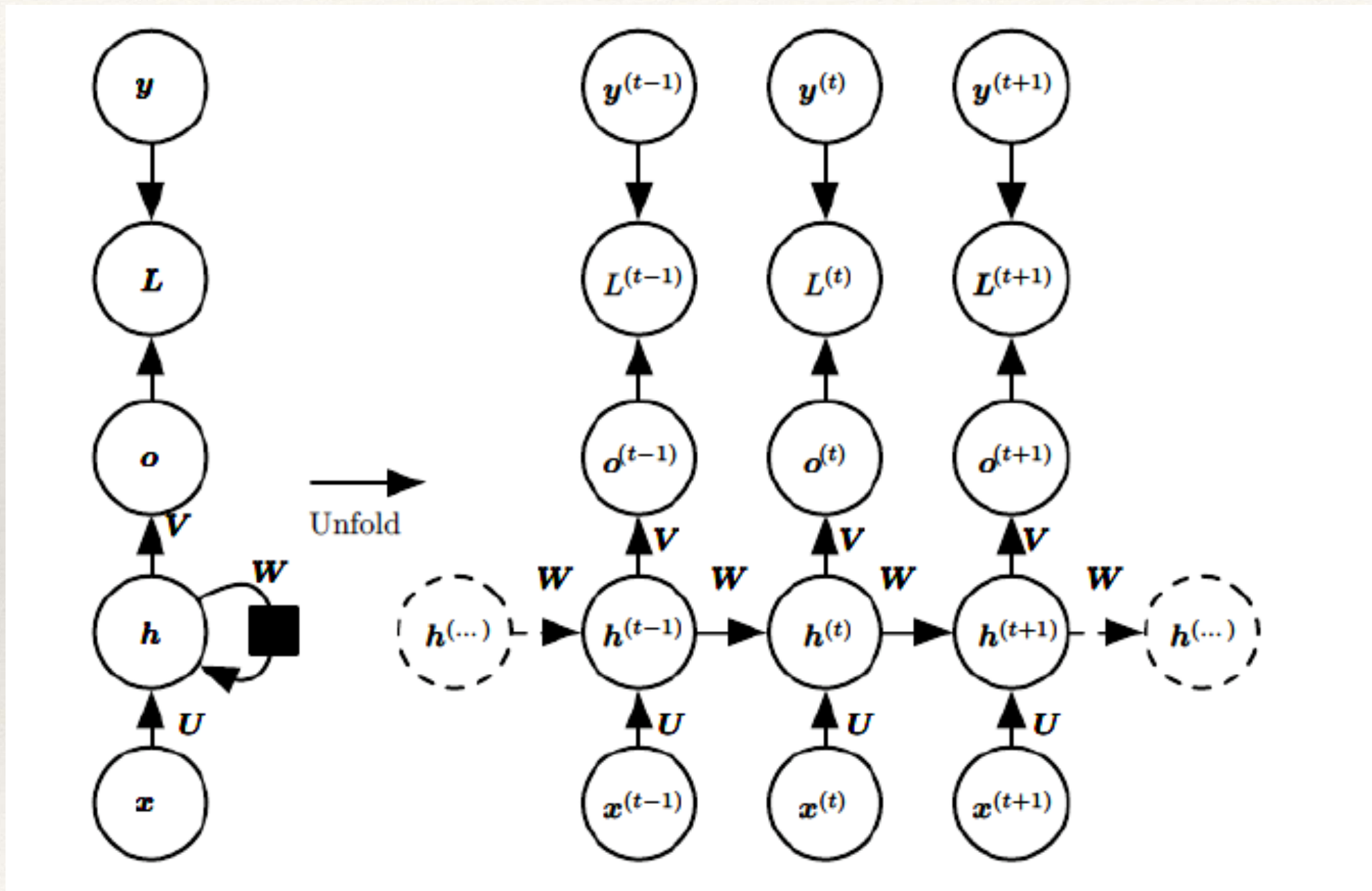
$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

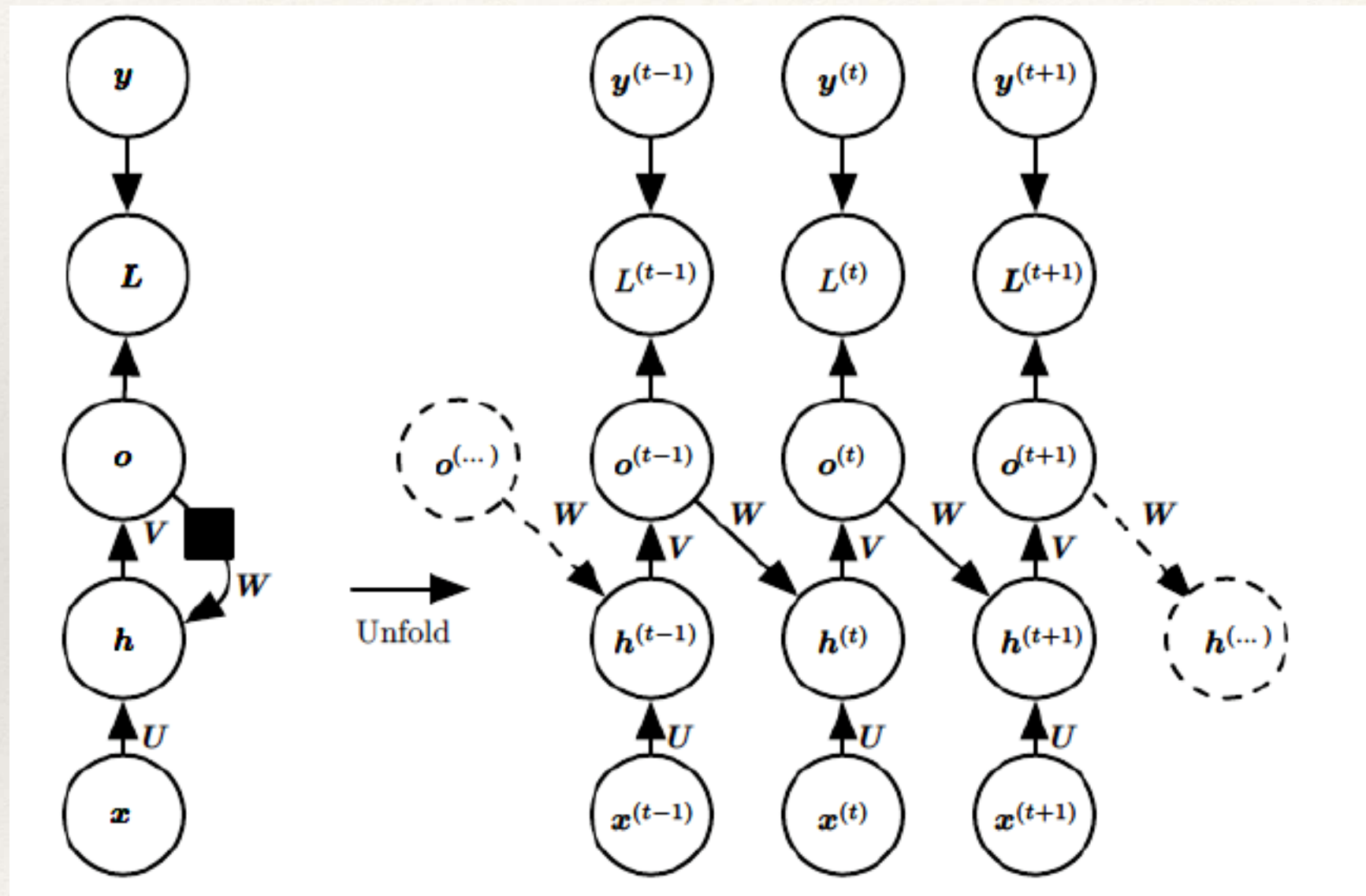
It combines the **forget** and **input** into a single **update gate**.
It also merges the cell state and hidden state. This is simpler than LSTM. There are many other variants too.

X,*: element-wise multiply

Standard Recurrent Networks

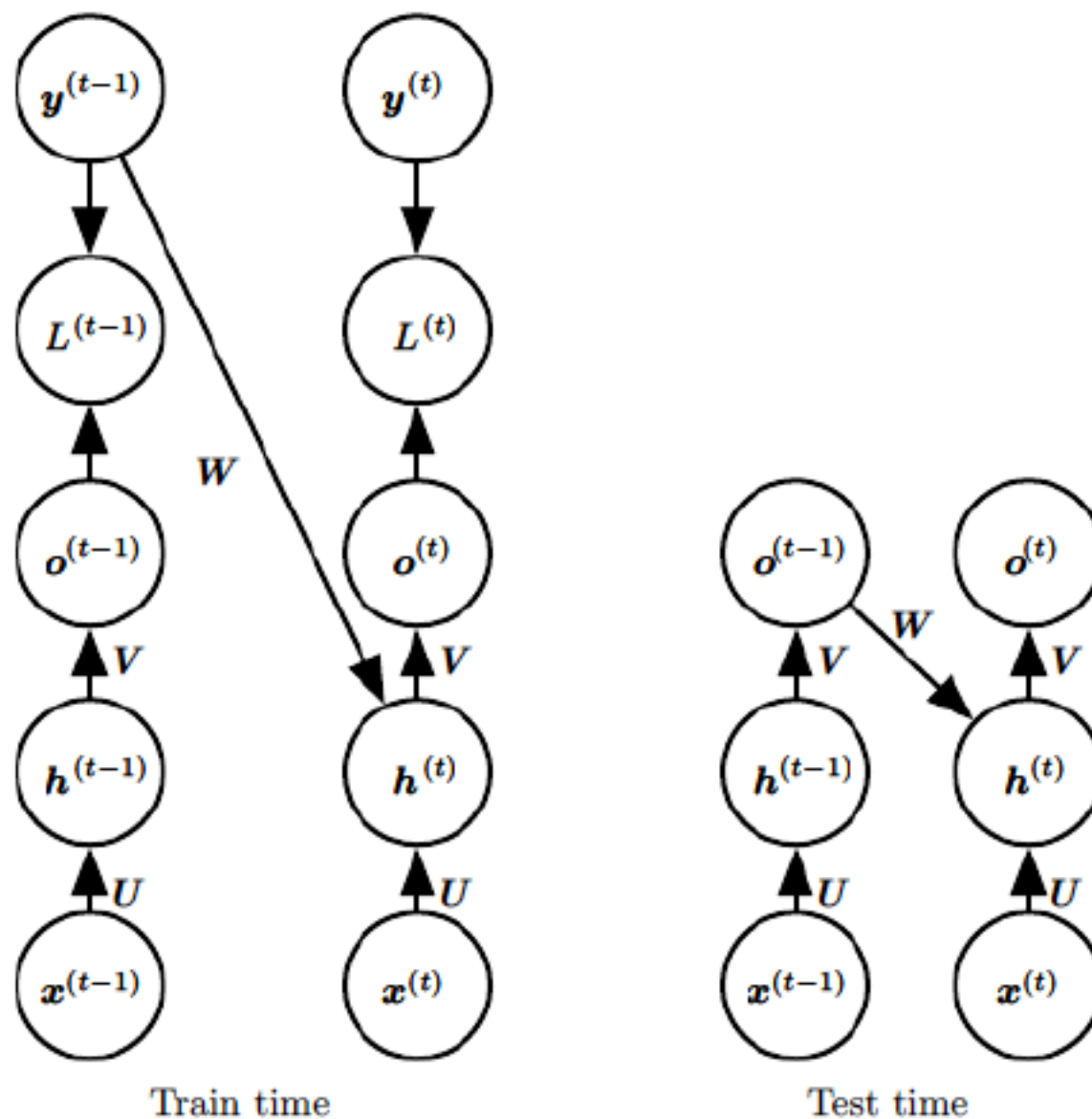


Other Recurrent Networks



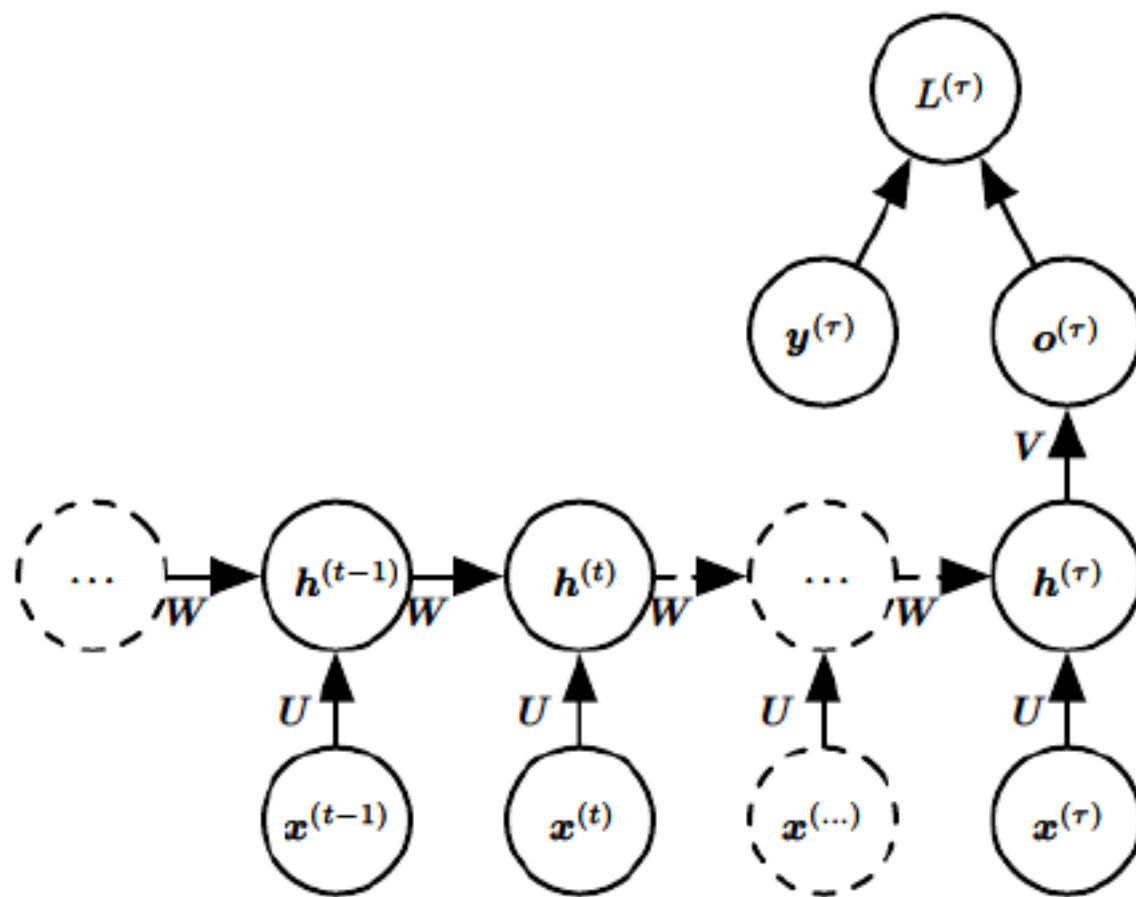
**Teacher
Forcing Networks**

Recurrent Networks



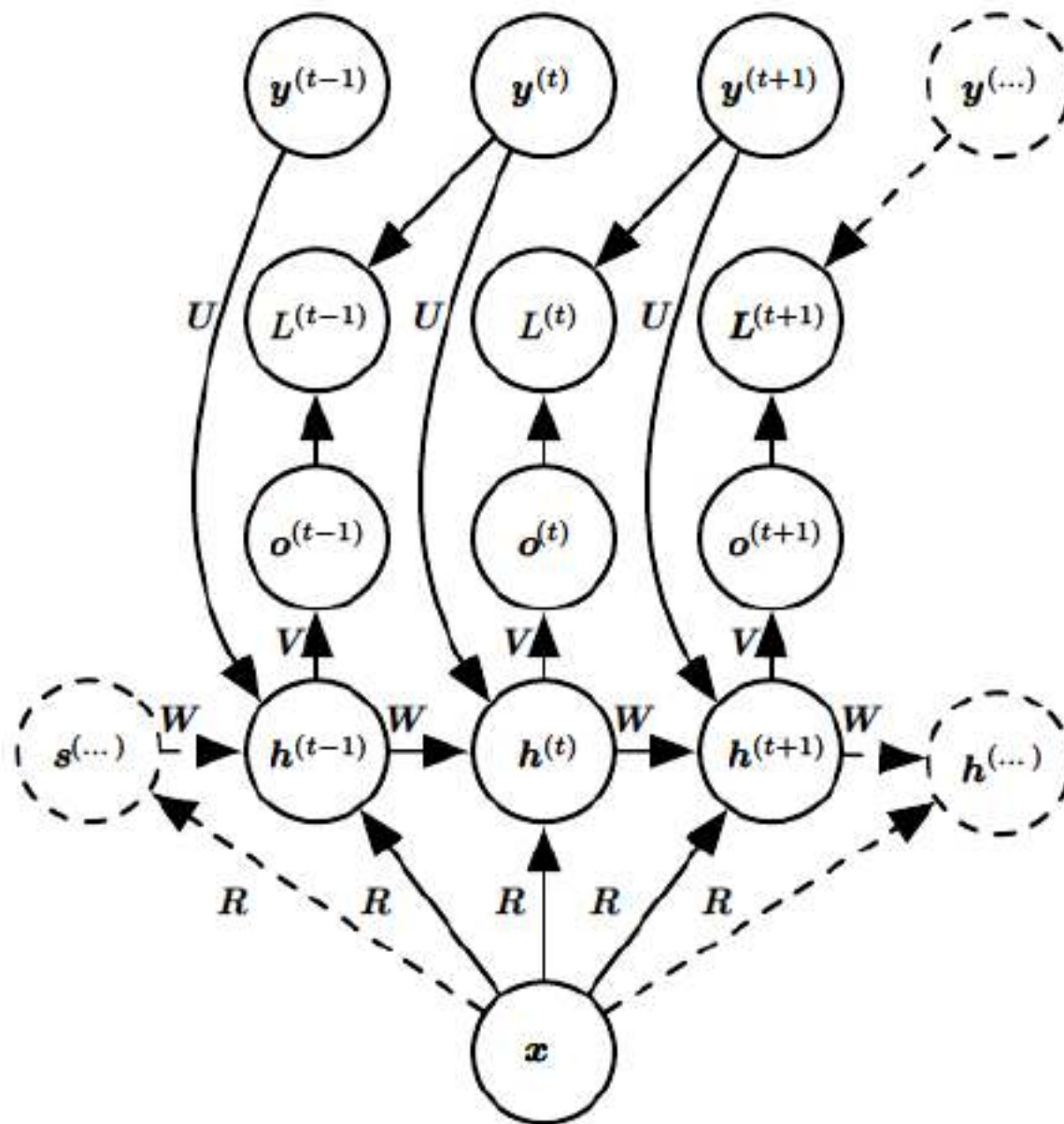
**Teacher
Forcing Networks**

Recurrent Networks



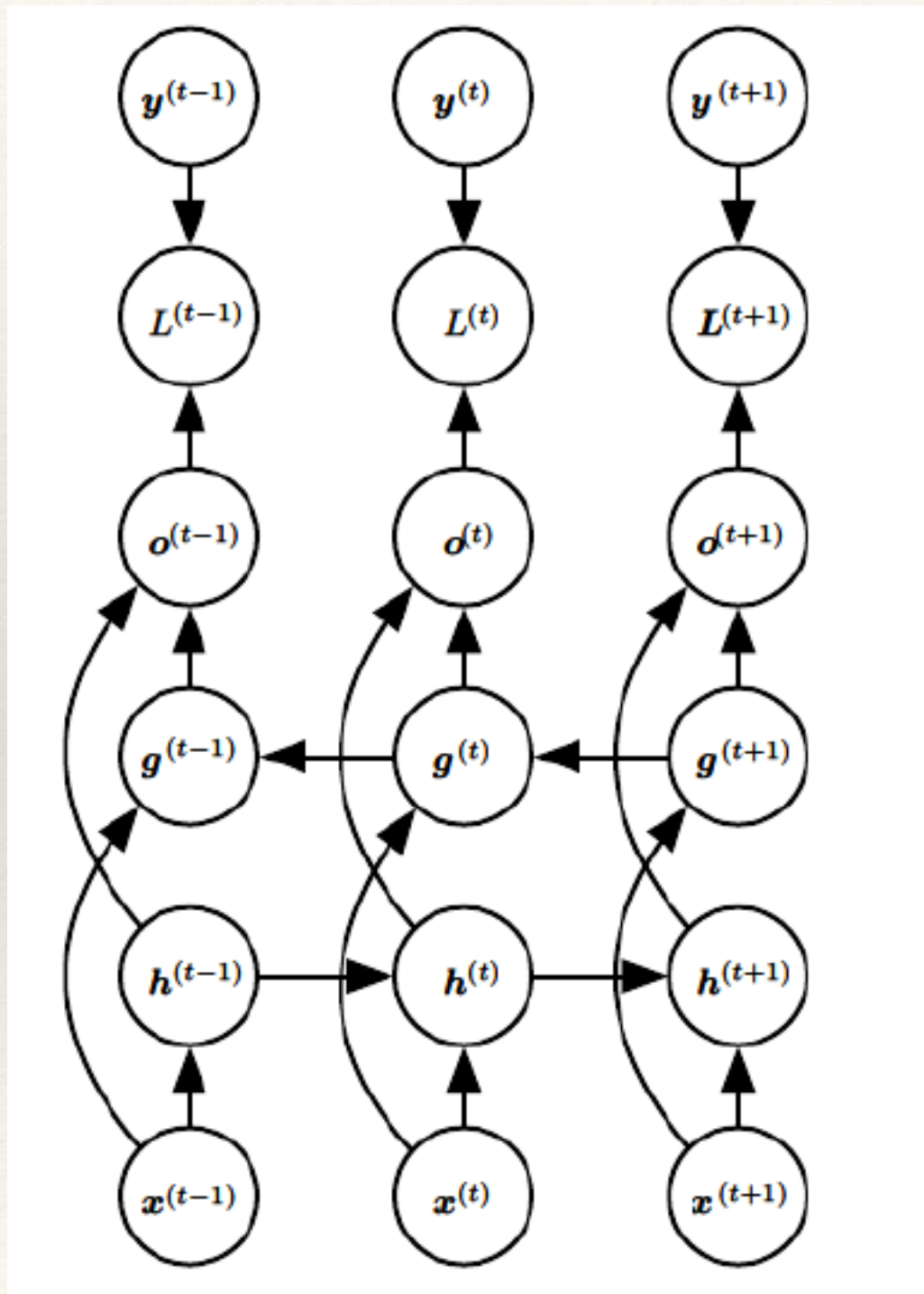
**Multiple Input
Single Output**

Recurrent Networks



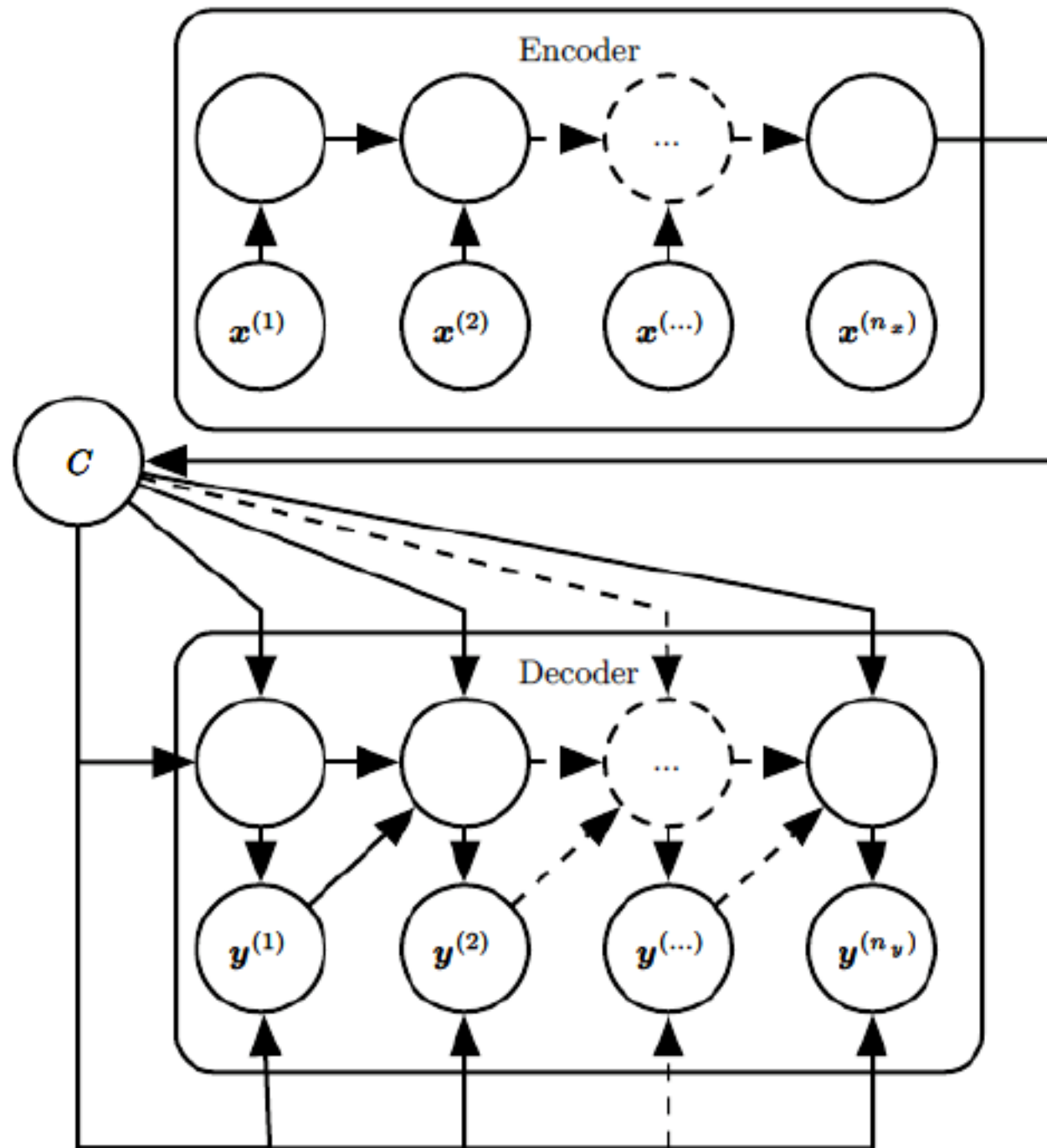
**Single Input
Multiple Output**

Recurrent Networks



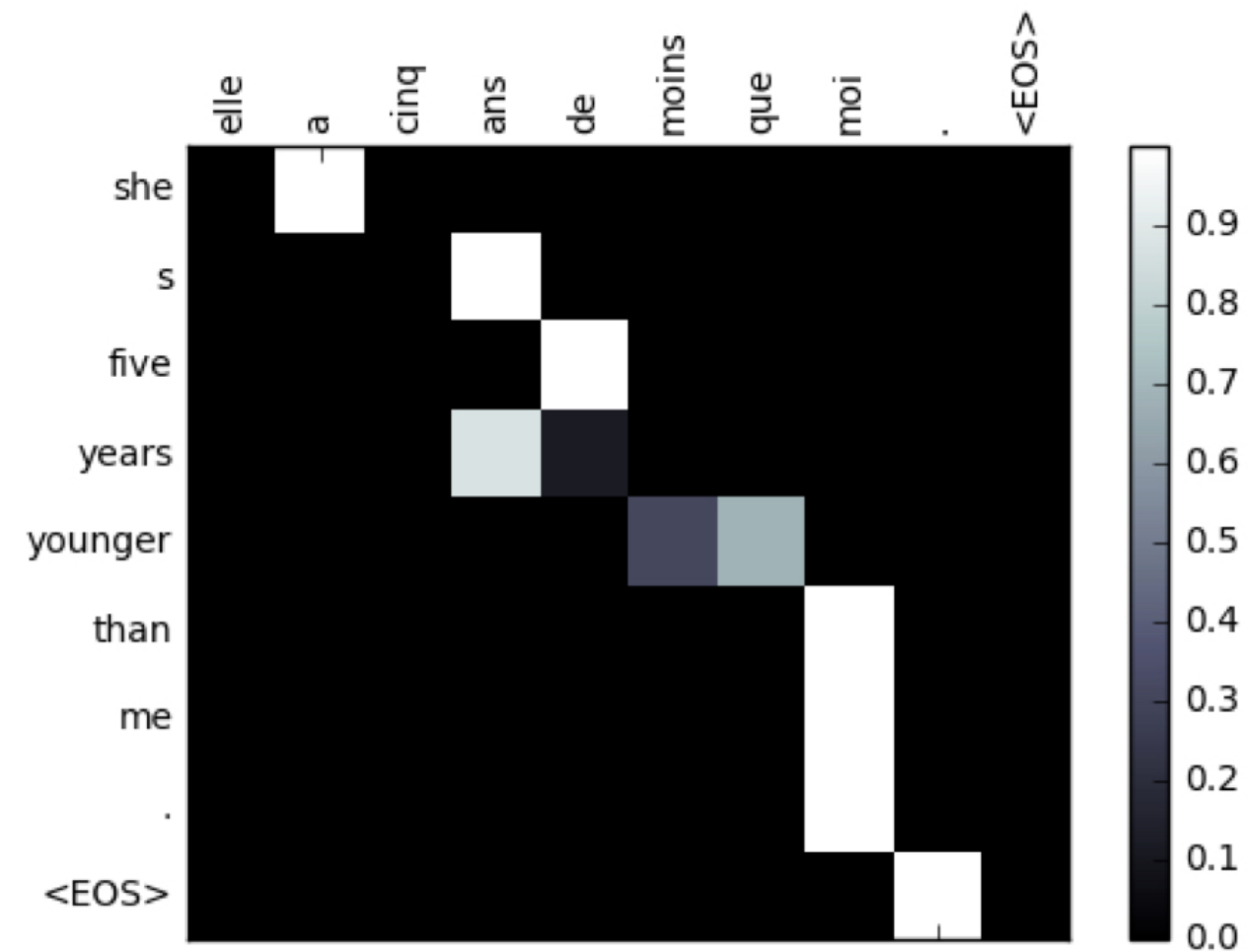
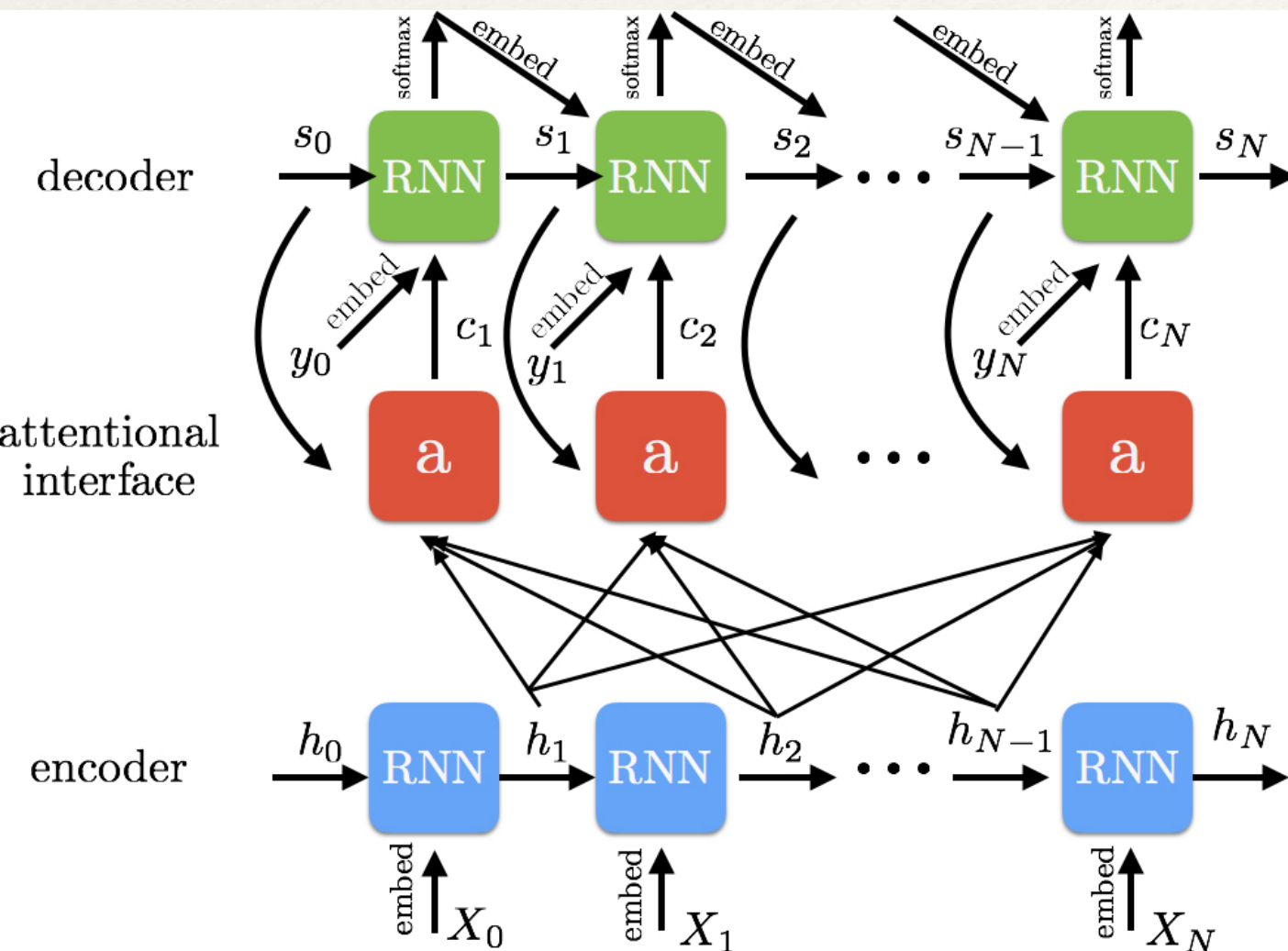
**Bi-directional
Networks**

Recurrent Networks



**Sequence to
Sequence
Mapping Networks**

Attention Models



Encoder - Decoder Networks with Attention

